

Funkcionális programozás

11. előadás

MÁRTON Gyöngyvér

Sapientia Egyetem, Matematika-Informatika Tanszék
Marosvásárhely, Románia
`mgyongyi@ms.sapientia.ro`

2026, tavaszi félév

Miről volt szó?

- ByteString-ek
- Haskell I/O műveletek: állománykezelés ByteString-ekkel

Miről lesz szó?

típusok és adatszerkezetek:

- rekord típusok
- algebrai adattípusok

Rekord típusok

- a korábban megismert adattípusok mellett új adattípusokat lehet definiálni, pl. rekord típusokat
- a definiáláshoz a **data** kulcsszót kell használni, ahol a választott név nagybetűvel kell kezdődjön:

```
data DiakT = DiakE String Int [Double]  
    deriving Show
```
- a DiakT lesz az új típusú adatszerkezet neve, amelyet típuskonstruktornak hívunk
- a DiakE azonosító értékkonstruktor lesz, ezt használjuk, amikor egy DiakT típusú adatnak értéket akarunk adni
- három mező definiálására került sor: a String, Int, [Double] a mezők típusát határozzák meg
- a **deriving**-ban megadott típusosztályok függvényei az újonnan definiált típus esetében is használhatóak lesznek

Rekord típusok

- a gyakorlatban a típus- és értékkonstruktorok gyakran **azonos nevet** kapnak, ez alapján újradefiniáljuk a fenti adatszerkezetet, és a továbbiakban így fogunk dolgozni:

```
data Diak = Diak String Int [Double]
    deriving(Show)
```

- ha értéket szeretnénk adni, akkor választanunk kell egy kisbetűvel kezdődő nevet:

```
diak = Diak "David" 5 [7.90,9.85,9.95,8.55]
```

- a Diak értékkonstruktor argumentumaként megadott értékek típusa **sorrendben** meg kell egyezzen a típus definiálásakor megadott mezők típusának sorrendjével,
- ha a sorrendet nem tartjuk be, akkor fordítási hibát kapunk:

```
diak = Diak 5 "David" [7.90,9.85,9.95,8.55]
```

```
...
```

- **Couldn't match expected type 'Int' with actual type ...**

Mezőkre való hivatkozás

- a mezőkre való hivatkozást mintaillesztéssel lehet megoldani
- a következő kiírja a diak-ban megadott nevet és a legnagyobb jegyet:

```
myShowDiak :: Diak -> String
myShowDiak k = nev ++ " " ++ show (maximum jegy)
  where
    Diak nev _ jegy = k
```

```
> putStrLn $ myShowDiak diak
David 9.95
```

- a mintaillesztést a következőképpen is meg lehet oldani:

```
myShowDiak_ :: Diak -> String
myShowDiak_ (Diak nev _ jegy) = nev ++ " " ++ show (maximum jegy)
```

Lista inicializálása

- egy Diak elemtípusú lista inicializálása, ahol vesszőt csak a listaelemek közé kell tenni, a mezőértékek között **tilos a vesszők** használata:

```
lsD = [ Diak "Ferenc" 1 [7.5,9.25],  
        Diak "Katalin" 2 [7.75,6.25,10,7.55],  
        Diak "Maria" 3 [6.5,8.25,9.33],  
        Diak "Zsuzsa" 4 [7.33,8.25,9.75],  
        Diak "David" 5 [7.90,9.85,9.95,8.55]]
```

- a myPrintDiakLs kiírja soronként a diákok neveit és a legnagyobb jegyüket:

```
myPrintDiakLs :: [Diak] -> IO()  
myPrintDiakLs = mapM_ (putStr . auxF)  
  where  
    auxF :: Diak -> String  
    auxF (Diak k1 k2 k3) = k1 ++ " " ++ show (maximum k3) ++ "\n"  
  
> myPrintDiakLs lsD  
Ferenc 9.25  
...
```

Típusszinonimák

- kifejezőbb ha az adatszerkezeteknek neveket adunk, azaz definiálásakor típusszinonimákat használunk:

```
type Nev = String
type Kod = Int
type Jegyek = [Double]
```

```
data DiakM = DiakM Nev Kod Jegyek
           deriving Show
```

```
myShowDiakM :: DiakM -> String
myShowDiakM k = nev ++ " " ++ show (maximum jegy)
  where
    DiakM nev _ jegy = k
```

```
diakM = DiakM "David" 5 [7.90,9.85,9.95,8.55]
```

```
> myShowDiakM diakM
"David 9.95"
```


Rekord típusok

- a Haskell nem engedi meg, hogy az ugyanolyan szerkezetű, de különböző nevű adatszerkezeteket összekeverjük,
- ha **összekeverjük** a korábban definiált:
 - `diak`, `diakM` azonosítókat,
 - a `Diak`, `DiakM` típusokat és
 - a `myShowDiak`, `myShowDiakM` függvényeket,

akkor futási hibát kapunk:

```
> myShowDiakM diak
```

```
... error:
```

```
> myShowDiak diakM
```

```
... error:
```

Rekord típus, mezőnevek

- adatszerkezetek definiálása történhet mezőnevek megadásával, ahol a mezőnevek mindig kisbetűvel kell kezdődjenek:

```
data DiakH = DiakH{  
    dNev  :: Nev,  
    dEv   :: Kod,  
    dJegyek :: Jegyek  
} deriving (Show)
```

- könnyebb a mezőneveken keresztül a hivatkozás, ahol nem számít a sorrend:

```
hasonlit :: DiakH -> DiakH -> DiakH  
hasonlit d1 d2  
    = if atlag (dJegyek d1) > atlag (dJegyek d2) then d1 else d2
```

- lehet mintaillesztést is használni, ahol számít a sorrend

```
hasonlit1 :: DiakH -> DiakH -> DiakH  
hasonlit1 h1@(DiakH a1 a2 a3) h2@(DiakH b1 b2 b3)  
    = if atlag a3 > atlag b3 then h1 else h2
```

```
atlag :: (Fractional a, Foldable t) => t a -> a  
atlag ls = sum ls / fromIntegral (length ls)
```

Rekord típus, mezőnevek

a fő program:

```
main2 = do
  let diakH1 = DiakH "Mari" 1 [4.5, 9.6, 6.5]
  let diakH2 = DiakH { dNev = "David",
                       dJegyek = [5.0, 8.0, 7.5, 10.0],
                       dEv = 2 }
  putStr "mezonevre valo hivatkozassal, "
  let res1 = hasonlit diakH1 diakH2
  putStrLn $ "a jobb atlaggal rendelkezo diak: " ++ dNev res1
  let res2 = hasonlit1 diakH1 diakH2
  putStr "mintaillesztessel, "
  putStrLn $ "a jobb atlaggal rendelkezo diak: " ++ dNev res2
```

Rekord típus, mezőnevek

- mező értékének a módosítása:

```
modosit :: DiakH -> Nev -> Kod -> DiakH  
modosit diakV nV eV = diakV {dNev = nV, dEv = eV}
```

- nem írható felül a diak1, létre kell hozni a diak2-t:

```
main3 = do  
    let diak1 = DiakH "Mari" 2 [7.5, 10, 8.75]  
    let diak2 = modosit diak1 "Anna" 2  
    putStrLn $ "DiakHA: " ++ show diak1  
    putStrLn $ "DiakHA1: " ++ show diak2
```

Rekord típusok, feladatok

Legyen a következő `Hallgato` típus, illetve a `hallgatoL` lista:

```
data Hallgato = Hallgato{  
    hNev :: String,  
    hJegy :: Double,  
    hEv  :: Int  
} deriving(Show)
```

```
hallgatoL = [Hallgato "Sari" 8.75 1, Hallgato "Mari" 4.25 1,  
             Hallgato "Feri" 3.5 2, Hallgato "Zsuzsi" 10.0 2,  
             Hallgato "Laci" 8.5 2, Hallgato "Lori" 7.5 2]
```

1. feladat

Számoljuk meg, hogy hány diáknak van átmenő jegye.

```
szamol :: [Hallgato] -> Int  
szamol ls = length $ filter (\k -> hJegy k > 4.5) ls  
  
> szamol hallgatoL  
4
```

Rekord típusok, feladatok

2. feladat

Válogassuk ki az első éves diákokat.

```
valogat :: Int -> [Hallgato] -> [Hallgato]
valogat e = foldr op []
  where
    op :: Hallgato -> [Hallgato] -> [Hallgato]
    op k rLs = if hEv k == e then k : rLs else rLs
```

halmazkifejezésekkel:

```
valogatLC :: Int -> [Hallgato] -> [Hallgato]
valogatLC e ls = [k | k <- ls, hEv k == e]
```

Rekord típusok, feladatok

A főprogram:

```
myShow :: Hallgato -> String
myShow k = hNev k ++ " " ++ show (hJegy k) ++ " " ++ show (hEv k)

main4 = do
    putStr "ev: "
    tStr <- getLine
    let e = read tStr :: Int
    --let nLs = valogat e hallgatoL
    let nLs = valogatLC e hallgatoL
    mapM_ (putStrLn . myShow) nLs
```

Rekord típusok, feladatok

3. feladat

Rendezzük a lista tartalmát választható kritérium alapján.

```
import Data.List
import Data.Ord

rendez :: Ord b => (a -> b) -> [a] -> [a]
rendez mezoNev ls = sortOn mezoNev ls

main5 = do
  putStr "Mi szerint szeretn;l rendezni? (nev, jegy, ev): "
  temp <- getLine
  let resL = case temp of
    "nev"  -> rendez hNev hallgatoL
    "jegy" -> rendez (Down . hJegy) hallgatoL
    "ev"   -> rendez hEv hallgatoL
    _      -> error "nincs ilyen mezon"
  mapM_ (putStrLn . myShow) resL
```


Rekord típusok, feladatok

4. feladat

Határozzuk meg a jegyek átlagát.

```
import Data.List

atlagHFoldL :: [Hallgato] -> Jegy
atlagHFoldL ls = ossz / db
  where
    (ossz, db) = auxAtlag ls
    auxAtlag :: [Hallgato] -> (Double, Double)
    auxAtlag ls = foldl' op (0.0, 0.0) ls
      where
        op (x1, x2) k = (x1 + hJegy k, x2 + 1)

> atlagHFoldL hallgatoL
7.083333333333333
```

Rekord típusok, szövegállományok

5. feladat

Egy cég egy adott alkalmazotról a következő adatokat tárolta el: név, év-jövedelem értékpárok. Írjunk egy Haskell-függvényt, amely egy megadott évre meghatározza minden alkalmazott jövedelmét.

```
type Jovedelem = (Int, Int)
data Alkalmazott = Alkalmazott {
    alkNev :: String,
    alkJovedelem :: [Jovedelem]
} deriving (Show, Read)
```

Feltételezzük, hogy a cég adatai az `alkalmazottData.txt` állományban vannak, pl:

```
[ Alkalmazott {alkNev = "KissCs",
  alkJovedelem = [(2012,8600),(2013,8900),(2014,8700)]},
  Alkalmazott {alkNev = "SzaboJ" ,
  alkJovedelem = [(2010,18000),(2011,21000),(2013,20000),(2014,24000)]},
  Alkalmazott {alkNev = "NagyS",
  alkJovedelem = [(2011,22000),(2012,23000),(2013,19000),(2014,24000)]},
  Alkalmazott {alkNev = "KovacsM",
  alkJovedelem = [(2013,9900),(2014,9800)]}]
```

Rekord típusok, szövegállományok

Az állomány szerkezete lehetővé teszi az egyszerű adatkiolvasást: a `temp`-be kerülő `String` típusú adatot egyetlen `read` segítségével át tudjuk alakítani `[Alkalmazott]` típusú értékke:

```
mainAlkalmazott :: IO ()
mainAlkalmazott = do
    temp <- readFile "alkalmazottData.txt"
    let lsAlk = (read :: String -> [Alkalmazott]) temp
    putStr "ev: "
    temp <- getLine
    let ev = read temp :: Int
    foJovedelem ev lsAlk

> mainAlkalmazott
ev: 2012
KissCs, 8600
SzaboJ, nincs jovedelem
...
```

Rekord típusok, szövegállományok

a következő `auxEv` ha lehetséges, akkor meghatározza az évre vonatkozó jövedelmet, ellenkező esetben `Nothing` lesz a kimeneti értéke:

```
auxEv :: Int -> [Jovedelem] -> Maybe Int
auxEv ev = foldr (op ev) Nothing
  where
    op ev (k1, k2) res =
      if ev == k1 then Just k2 else res

> auxEv 2010 [(2012,8600),(2013,8900),(2014,8700)]
Nothing
```

Rekord típusok, szövegállományok

a foJovedelem-ben minden egyes alkalmazott esetében meghívásra kerül az auxEv függvény, amelynek az eredményét egy case-ben elemezzük

```
foJovedelem :: Int -> [Alkalmazott] -> IO()
foJovedelem ev = mapM_ (auxF ev)
  where
    auxF :: Int -> Alkalmazott -> IO()
    auxF ev k =
      case res of
        Just x -> putStrLn $ alkNev k ++ ", " ++ show x
        Nothing -> putStrLn $ alkNev k ++ ", nincs jovedelem"
  where
    res = auxEv ev (alkJovedelem k)
```

Rekord típusok, szövegállományok

6. feladat

Határozzuk meg egy megadott évre a maximális jövedelmet és azon alkalmazottakat, akiknek maximális volt a jövedelme.

- a feladat megoldásához használni fogjuk a korábban megadott `Jovedelem` típust és `auxEv` függvényt,
- feltételezve, hogy ezek az `feladatJov.hs`-ben vannak megadva, módosítsuk ennek a tartalmát, az első sorba írjuk be:

```
module FeladatJov where
```

- a mostani feladathoz tartozó definíciókat, illetve függvényeket egy másik fileba fogjuk írni, amelynek első sorába írjuk be:

```
import qualified FeladatJov as E11
```

- a mostani feladathoz a következő típust fogjuk használni:

```
data Alkalmazott = Alkalmazott String [E11.Jovedelem]  
    deriving (Show, Read)
```

- vegyük észre hogy ez más mint ahogy az `feladatJov.hs`-ben definiál `Alkalmazott` típus!

Rekord típusok, szövegállományok

- feltételezzük, hogy a cég adatai a következő formában, az `alkalmazott.txt` állományban vannak, pl:

```
KissCs [(2012,8600),(2013,8900),(2014,8700)]
SzaboJ [(2010,18000),(2011,21000),(2013,20000),(2014,24000)]
NagyS [(2011,22000),(2012,23000),(2013,19000),(2014,24000)]
KovacsM [(2013,9900),(2014,9800)]
```

- vegyük észre, hogy a korábbi `alkalmazottData.txt` állomány szerkezete más volt,
- `alkalmazott.txt` állomány beolvasását **másképp kell végezni**:
 - soronként kell a megfelelő listaelemeket létrehozni,
 - a jövedelem értékekre fogjuk alkalmazni a `read` függvényt.
- szükség lesz még a következő importok-ra is:

```
import Data.List ( maximumBy )
import Data.Ord ( comparing )
```

Rekord típusok, szövegállományok

- a beolvasást a `myReadFile` végezi,
- a `foMaxJovedelem`-ben határozzuk meg a maximum elemeket, illetve itt végezzük az eredményeket kiíratását,
- a feladat főfüggvénye a következő:

```
mainAlkalmazott :: IO ()
mainAlkalmazott = do
    lsAlk <- myReadFile "alkalmazott.txt"
    --print lsAlk
    putStr "ev: "
    temp <- getLine
    let ev = read temp :: Int
    foMaxJovedelem ev lsAlk
```

```
> mainAlkalmazott
ev: 2014
maximalis jovedelem: 24000
SzaboJ
NagyS
```


Rekord típusok, szövegállományok

- a `myReadFile` soronként fogja, a `lines` függvényt alkalmazva feldolgozni a `temp`-el jelölt állománytartalmat
- a sorokat a `words`-al bontjuk szavakra

```
myReadFile :: FilePath -> IO [Alkalmazott]
myReadFile nev = do
  temp <- readFile nev
  let ls = map auxF $ lines temp
  return ls
  where
    auxF ls = Alkalmazott nevA jovA
    where
      [nevA, tJovA] = words ls
      jovA = (read :: String -> [FJ.Jovedelem]) tJovA
```

Rekord típusok, szövegállományok

- a `maximumAlk` kimenete **egy tuple** típusú érték:
 - a **`maxNev`** annak a személynek a nevét jelöli, akinek maximális volt a jövedelme az adott évben
 - a **`max`** a maximális jövedelmet jelöli, amely `-1` lesz, ha az adott évben senkinek sincs jövedelem nyilván tartva,
- az **`mLs`**-ben azoknak az alkalmazottaknak a nevei kerülnek, akiknek az adott évben maximális volt a jövedelme

```
foMaxJovedelem :: Int -> [Alkalmazott] -> IO()
foMaxJovedelem ev ls = do
  let (maxNev, max) = maximumAlk ev ls
  let mLs = [nevA | Alkalmazott nevA jovA <- ls,
                    E11.auxEv ev jovA == Just max]
  case max of
    -1 -> putStrLn "nincs jovedelem"
    _   -> do
      putStrLn $ "maximalis jovedelem: " ++ show max
      mapM_ putStrLn mLs
```

Rekord típusok, maximum keresés

- a maximum meghatározása a beépített `maximumBy`-al történik,
- az `auxF` az év évben a `kAlk` jövedelmét határozza meg, a `map (auxF ev) ls`-al pedig minden alkalmazott jövedelme meghatározásra kerül,
- egy alkalmazott adott évbeli jövedelemértékének a kiválasztását az `feladatJov.hs` található `auxEv` végezi,

```
maximumAlk :: Int -> [Alkalmazott] -> (String, Int)
maximumAlk ev ls = maximumBy (comparing snd) $ map (auxF ev) ls
```

```
auxF :: Int -> Alkalmazott -> (String, Int)
auxF ev kAlk = (nevA, k)
  where
    Alkalmazott nevA jovA = kAlk
    temp = E11.auxEv ev jovA
    k = case temp of
      Nothing -> -1
      Just x -> x
```

```
> auxF 2011 $ Alkalmazott "KissCs" [(2012,8600),(2013,8900),(2014,8700)]
("KissCs",-1)
> auxF 2014 $ Alkalmazott "KissCs" [(2012,8600),(2013,8900),(2014,8700)]
("KissCs",8700)
```

Rekord típusok, maximum keresés

- a maximum keresés hatékony megoldását a `maximumAlk_` végzi, mert ez a függvény az alkalmazott lista egyszeri bejárásával határozza meg az összes maximum elemet:

```
maximumAlk_ :: Int -> [Alkalmazott] -> ([String], Int)
maximumAlk_ ev ls = (mLs, max)
  where
    (mLs, max) = foldr op res ls
    res = ([], -1)
    op :: Alkalmazott -> ([String], Int) -> ([String], Int)
    op kAlk t
      | k == m = (nevA : nevLs, m)
      | k < m = t
      | k > m = ([nevA], k)
      where
        (nevLs, m) = t
        Alkalmazott nevA jovA = kAlk
        temp = FJ.auxEv ev jovA
        k = case temp of
              Nothing -> -1
              Just x -> x
```

Rekord típusok, maximum keresés

- a hívó `foMaxJovedelem_` is módosul a következők szerint:

```
foMaxJovedelem_ :: Int -> [Alkalmazott] -> IO()
foMaxJovedelem_ ev ls = do
    let (mLs, max) = maximumAlk_ ev ls
    case max of
        -1 -> putStrLn "nincs jovedelem"
        _   -> do
            putStrLn $ "maximalis jovedelem: " ++ show max
            mapM_ putStrLn mLs
```

Algebrai adattípusok

- több értékkonstruktor is lehetséges
- a legismertebb algebrai adattípus a `Bool`, amely két értékkonstruktorral rendelkezik, a `True`-val és a `False`-szal:
`data Bool = False | True`
- az értékkonstruktorokat `|`-al kell elválasztani
- algebrai adattípust mi is definiálhatunk, például létrehozhatunk egy saját `Bool` típust, amelyet egy természetes szám párosságának a vizsgálatához ugyanúgy lehet használni, mint `Bool`-t:

```
data MyBool = Igaz | Hamis
    deriving (Show)
```

```
parosT :: Integral a => a -> MyBool
parosT n = if even n then Igaz else Hamis
```

Algebrai adattípusok

- a következő példában a Szemely típusnak két értékkonstruktor lesz, az első, a Diak három, míg a második, a Tanár két mezővel rendelkezik
- az értékkonstruktorokat írhatjuk egymás mellé, de tördelve, külön sorba is meg lehet adni őket

```
data Szemely = Diak String String Double
             | Tanar String String
  deriving (Show)
```

- egy Szemely elemtípusú lista kezdőértéke a következő lehet:

```
lsSz :: [Szemely]
lsSz = [Diak "Laci" "ELTE" 4.5, Tanar "Feri" "ELTE",
        Tanar "Mari" "Sapientia", Diak "Lori" "Sapientia" 7.5,
        Diak "Sari" "Sapientia" 8.75, Tanar "Zsuzsi" "ELTE"]
```

Algebrai adattípusok

7. feladat

Írjunk egy Haskell függvényt, amely egy `Szemely` típusú lista esetében meghatározza a 4.5-nél nagyobb jeggyel rendelkező diákok számát:

```
szamolDiak :: [Szemely] -> Int
szamolDiak ls = length $ filter auxF ls
  where
    auxF (Tanar _ _) = False
    auxF (Diak _ _ jegy) = jegy > 4.5
```

```
> szamolDiak lsSz
2
```

Mintaillesztéssel választottuk külön a Tanar, illetve a Diak értékekkel való műveletvégzést.

Algebrai adattípusok

A feladat megoldható a `foldl'` függvény használatával is, a kódsor a következő:

```
import Data.List (foldl')
szamolDiak_ :: [Szemely] -> Int
szamolDiak_ = foldl' op 0
  where
    op res k = case k of
      Tanar _ _ -> res
      Diak _ _ jegy -> if jegy > 4.5 then 1 + res
                       else res
```

Algebrai adattípusok

A következőkben előbb négy típuszinonimát definiálunk, majd egy `BankSzamla` típust, három értékkonstruktorral, amelyekben használjuk a típuszinonimaként megadott típusokat:

```
type KartyaSz = String
type Tulajdonos = String
type Cim = [String]
type FelhasznaloID = Int
```

```
data BankSzamla = BankKartya KartyaSz Tulajdonos Cim
                | Keszpenz
                | Szamla FelhasznaloID
                deriving (Show, Eq)
```

Algebrai adattípusok

Konstans értékeket a következőképpen hozhatunk létre:

```
sz1 = BankKartya "12321" "Kiss Antal" ["Mvh", "Romania"]
sz2 = Keszpenz
sz3 = Szamla 12
sz4 = BankKartya "54321" "Nagy Antal" ["Kv", "Romania"]
sz5 = BankKartya "98765" "Beres Antal" ["Mvh", "Romania"]
sz6 = Szamla 13
```

```
lsBsz = [sz1, sz2, sz3, sz4, sz5, sz6]
```

8. feladat

Írjunk egy Haskell-függvényt, amely kiválogatja egy `BankSzamla` elemtípusú listából azokat a személyeket, akiknek értékkonstruktora `BankKartya`.

```
valogatSz :: [BankSzamla] -> [Tulajdonos]
valogatSz [] = []
valogatSz (k : ve) = case k of
    BankKartya _ tu _ -> tu : valogatSz ve
    _ -> valogatSz ve
```

Algebrai adattípusok

A függvényt még kétféleképpen is megadjuk: a `valogatSzF` a `foldr` függvényt alkalmazza. A `foldr` helyett alkalmazhattuk volna valamelyik `foldl` változatot is.

```
valogatSzF :: [BankSzamla] -> [Tulajdonos]
valogatSzF = foldr op []
  where
    op k res = case k of
      BankKartya _ tu _ -> tu : res
      _ -> res
```

A `valogatSzH` halmazműveleteket használ:

```
valogatSzH :: [BankSzamla] -> [Tulajdonos]
valogatSzH ls = [tu | BankKartya _ tu _ <- ls]

> valogatSz lsBsz
["Kiss Antal","Nagy Antal","Beres Antal"]
```

Algebrai adattípusok

9. feladat

Írjunk egy Haskell-függvényt, amely külön listákba teszi a BankKartya, a Keszpenz és a Szamla értékkonstruktorokra vonatkozó adatokat. A Keszpenz esetében csupán számoljuk meg, hogy hány készpénzkifizetés van.

```
valogatBK :: [BankSzamla] -> ([BankSzamla], [Int], [BankSzamla])
valogatBK ls = auxV ls [] [0] []
  where
    auxV [] bLs kLs sLs = (bLs, kLs, sLs)
    auxV (k : ve) bLs kLs sLs = case k of
      BankKartya {} -> auxV ve (k : bLs) kLs sLs
      Keszpenz -> auxV ve bLs [1 + head kLs] sLs
      Szamla _ -> auxV ve bLs kLs (k : sLs)
```

```
> valogatBK lsBs
([BankKartya "98765" "Beres Antal" ["Mvh",...
```

A case-ben, a BankKartya értékkonstruktor esetében a `_ _ _` helyett a `{}` szimbólumokat használjuk, mert a `->` jobb oldalán egyetlenegy mezőértékkel sincs műveletvégzés.

Algebrai adattípusok

Az elegánsabb kiíratáshoz `mapM_-et` használunk, amelyet külön-külön alkalmazunk a tuple elemekre, amelyeket a `valogatBK` függvény visszatérési értékeként kaptunk meg:

```
foValogat :: [BankSzamla] -> IO ()
foValogat ls = do
  let (t1, t2, t3) = valogatBK ls
  putStrLn "Bankkartya adatok:"
  mapM_ print t1
  putStrLn "Keszpenzkifizetesek szama: "
  mapM_ print t2
  putStrLn "Szamla adatok:"
  mapM_ print t3

> foValogat lsBsz
Bankkartya adatok:
BankKartya "98765" "Beres Antal" ["Mvh","Romania"]
...
```